

Propeller 2 • Application Note P2AN001

Measure an Absolute Voltage in Microvolts on a P2 Pin

No external ADC — a single-pin instrumentation ADC and a catalog of techniques

November 2026

Version 1.0.6

What You'll Build

Read an absolute voltage, in microvolts, on a single P2 pin — with no external converter.

A single-pin instrumentation ADC that streams a live microvolt reading to a DEBUG scope window, verified on the bench with one jumper wire — then a small catalog of techniques you choose among by what your project needs:

- **Base build** — single-pin absolute microvolts (the foundation)
- **Three pins** — constant impedance for lower noise in the same time
- **Filter cascade** — every rate-vs-resolution trade available at once
- **Range extension** — read voltages above 3.3 V with one resistor
- **Mains averaging** — null 50/60 Hz pickup for free
- **The ceiling** — eight channels via a bytecode interpreter (reference)

Applies to: P2 (Propeller 2) silicon • Spin2 / PNut, current release • P2 Edge or P2 Eval board, plus one jumper wire.

Iron Sheep Productions, LLC

P2 Knowledge Base Project

Abstract

Every P2 I/O pin contains a sigma-delta ADC. With nothing more than the smart pin, three internal reference measurements, and a few lines of code, you can read an **absolute** voltage on a pin — in microvolts — without any external converter. This note shows how to build that measurement once as a clean, reusable base, then how to choose among a small set of techniques to trade resolution, channel count, and sample rate for whatever your project actually needs.

What you'll build: a single-pin instrumentation ADC that streams a live microvolt reading to a DEBUG scope window, verified on the bench with one jumper wire.

Prerequisites & Hardware

You should already be comfortable with:

- Configuring a smart pin (WRPIN / WXPIN / WYPIN, raising DIR to enable it).
- Starting code in another cog with COGINIT.
- The P2's ADC modes at the mechanism level. This note *uses* the ADC; it does not re-teach it. For how the sigma-delta front end, the SINC filters, the gain ladder, and the internal references work, see the **I/O & Smart Pins User Guide**, Chapter 16 — in particular §16.3 (mode `P_ADC`) and its "Ratiometric Absolute-Voltage Instrumentation," which is the mechanism this note applies.

Hardware:

- A P2 Edge or P2 Eval board.
- **One jumper wire** between two free pins. This note uses pin 33 as a test-signal source and pin 32 as the pin under measurement; any two adjacent free pins work.
- Nothing else. The test signal is generated on-chip by the P2's own DAC, so the whole note runs on the bench with no instruments. A precision external reference and a meter come in only when you want to characterize *absolute* accuracy rather than confirm the build runs.

The Idea

An ADC reading is only ever a comparison. The P2's pin ADC compares the voltage at the pin against the chip's own internal references — but those references are not perfect, fixed quantities: they drift with the supply voltage and with temperature. If you read the pin alone, you get a number that *moves* even when the input doesn't.

The way out is to measure the references too, in the same units, at nearly the same moment, and then ask a ratiometric question:

> Where does the pin sit, as a fraction, between the internal ground reference and the internal supply reference?

The P2 lets you point one pin's ADC at three different things in turn: its internal **ground reference** (`P_ADC_GIO`), its internal **supply reference** (`P_ADC_VIO`), and the **external pin** itself. Read all three, and the absolute voltage falls out of a single ratio:

microvolts = (pin − GIO) / (VIO − GIO) × 3,300,000

Because every term is measured by the *same* ADC through the *same* path, the supply and temperature drift that would corrupt a lone pin reading divides out. That is what makes the result *absolute* rather than merely *relative*. Reading only two of the three (the common shortcut of pin-versus-float) is far noisier — there is no substitute for reading all three.

How It Works (in brief)

The mechanism lives in the **I/O & Smart Pins User Guide**, Chapter 16. Three points from there are all you need to follow the build:

- **SINC2 filtering mode.** The ADC runs in mode `P_ADC` with the X register set to `%01_0111` — SINC2 *filtering*, with a 128-clock sample period. In this mode the smart pin maintains a running accumulator; software reads it each period and takes the **difference** between consecutive reads to recover a sample. Summing many of these per-period differentials is what grows resolution into the microvolt range. (See §16.3, "SINC2 Filtering Mode," and the note on accumulating differentials beyond 14 bits.)
  At 200 MHz a 128-clock period gives a raw sample rate of `200_000_000 / 128 = 1,562,500` samples per second, so a sum of a few hundred differentials still completes in well under a millisecond.
- **The three-source rotation.** Switching the ADC's input among GIO, VIO, and the pin is just a live WRPIN with a different input-select constant — the smart pin keeps running. The code rotates through `P_ADC_GIO`, `P_ADC_VIO`, and `P_ADC_1X` (the external pin at 1× gain), keeping a separate accumulator for each. Both `P_ADC_GIO` and `P_ADC_VIO` reference the pin's *own* supply domain: the P2 powers its I/O pins in isolated groups of four (P0–3, P4–7, …, P60–63), and a pin's ADC measures the VIO/GIO of its own group. For a single pin that is automatic; when you tie several pins together it constrains which pins to use (see Pitfalls).
- **The source-switch flush.** Each time you change the input source, the **first three samples are contaminated** — two for the SINC2 filter to decimate the step through, one for the analog front end to settle. The **fourth** sample is the first clean one, so the measurement routine discards three and then sums the next eight. (This is distinct from the one-time startup warm-up; see §16.3, "Startup warm-up and source-switch flush are two different discards.")

How far the *absolute* number can be trusted is bounded by the front end itself — most importantly a matched-resistor error floor that no amount of averaging removes. Those limits are covered in §16.8 and revisited in this note's Pitfalls.

The Test Harness

Before the ADC build, set up two things you will reuse for every technique in this note: a way to **see** the reading, and a known signal to **read**. Defining them once here keeps each later recipe down to just its ADC core.

Seeing the reading. A DEBUG `SCOPE` window, scaled directly in microvolts (0 to 3,300,000), turns the live stream into a trace you can watch settle.

Generating a known signal. Rather than wire up an external reference, drive a second pin with the P2's own 16-bit DAC and jumper it across to the pin under measurement. This is a *functional* (bench-loopback) check — it proves the code end to end and gives you a value you can predict, but it does not certify *absolute* accuracy (the DAC and the ADC share the same chip's imperfections). For absolute work you replace this loopback with a precision external reference; the code under test does not change.

The harness drives the DAC to half scale (`TEST_UV = 1_650_000`, about 1.65 V), so a correct build reads back **near 1,650,000 µV**.

```
PRI start_test_signal() | dac16
  ' Drive DAC_PIN to a steady, known voltage so a jumper to ADC_PIN
  ' gives the ADC something real to read with no external gear. This
  ' is a functional (loopback) check: it proves the code, not accuracy.
  dac16 := muldiv64(TEST_UV, $FFFF, FULLSCALE_UV)    ' uV -> 16-bit DAC
  WRPIN(DAC_PIN, P_DAC_DITHER_PWM | P_DAC_124R_3V | P_OE)
  WXPIN(DAC_PIN, $100)       ' 256-clock DAC period (minimum)
  PINH(DAC_PIN)              ' enable the smart pin first
  WYPIN(DAC_PIN, dac16)      ' then set the output level

PRI open_scope()
  DEBUG(`SCOPE uV size 600 400 samples 256 rate 1 dotsize 2 pos 100 100)
  DEBUG(`uV 'uV' 0 3_300_000 360 20 15 MAGENTA)
```

★ **Tip:** The DAC here uses 16-bit PWM dither, so its output carries a small switching component. The integrating ADC averages that away over its sample window, which is exactly why a dithered DAC is a fine loopback source — but it is also why this is a functional check, not an accuracy benchmark.

The Base Build — Single-Pin Instrumentation ADC

This is the centerpiece: a complete, runnable instrumentation ADC on one pin. A PASM2 cog does the timing-critical work — rotating the three sources, taking the SINC2 differentials, discarding the flush samples, and summing only the clean samples — because that inner loop runs at the sample rate. The cog hands three accumulated sums back to Spin2 through a small mailbox, and Spin2 does the one ratiometric divide (which is *not* timing-critical) with MULDIV64, exactly the calculation the I/O & Smart Pins User Guide gives in §16.3.

The whole program compiles clean under `pnut-ts`:

```
CON
  _clkfreq = 200_000_000

  ADC_PIN  = 32              ' pin under measurement (jumper from DAC_PIN)
  DAC_PIN  = 33              ' loopback test-signal source

  FULLSCALE_UV = 3_300_000   ' VIO full scale, in microvolts
  TEST_UV      = 1_650_000   ' loopback target: ~half scale (~1.65 V)

  SUMS = 64                  ' source rotations summed per reading
                             '   raise for resolution, lower for rate

VAR
  LONG mbox[4]               ' cog<->Spin2: [0]Gio [1]Vio [2]Sig [3]flag

PUB main() | gioRef, vioRef, pinRef, uv
  start_test_signal()        ' harness: drive a known voltage onto the pin
  open_scope()               ' harness: DEBUG SCOPE, scaled in microvolts
  COGINIT(NEWCOG, @adc_engine, @mbox)        ' start the ADC cog

  repeat
    repeat until mbox[3]     ' wait for a fresh reading
    gioRef := mbox[0]
    vioRef := mbox[1]
    pinRef := mbox[2]
    mbox[3] := 0             ' acknowledge; release the cog

    ' Ratiometric: where does the pin sit between GIO and VIO?
    ' (mechanism: I/O & Smart Pins User Guide, Ch. 16)
    uv := muldiv64(pinRef - gioRef, FULLSCALE_UV, vioRef - gioRef)

    DEBUG(`uV `(uv))         ' stream the reading to the scope

PRI start_test_signal() | dac16
  ' Drive DAC_PIN to a steady, known voltage so a jumper to ADC_PIN
  ' gives the ADC something real to read with no external gear. This
  ' is a functional (loopback) check: it proves the code, not accuracy.
  dac16 := muldiv64(TEST_UV, $FFFF, FULLSCALE_UV)    ' uV -> 16-bit DAC
  WRPIN(DAC_PIN, P_DAC_DITHER_PWM | P_DAC_124R_3V | P_OE)
  WXPIN(DAC_PIN, $100)       ' 256-clock DAC period (minimum)
  PINH(DAC_PIN)              ' enable the smart pin first
  WYPIN(DAC_PIN, dac16)      ' then set the output level

PRI open_scope()
  DEBUG(`SCOPE uV size 600 400 samples 256 rate 1 dotsize 2 pos 100 100)
  DEBUG(`uV 'uV' 0 3_300_000 360 20 15 MAGENTA)

DAT             ORG

adc_engine
                MOV     pa, #ADC_PIN
                OR      pa, #%001_000000  ' SE1 = this pin's IN rising
                SETSE1  pa
```

continues on next page →

↪ continued from previous page

```
                FLTL    #ADC_PIN          ' reset the smart pin
                WRPIN   SetSig, #ADC_PIN  ' ADC mode, pin selected
                WXPIN   #%01_0111, #ADC_PIN   ' SINC2 filter, 128 clocks
                DIRH    #ADC_PIN          ' enable the smart pin

loop            MOV     Gio, #0           ' clear the three accumulators
                MOV     Vio, #0
                MOV     Sig, #0
                MOV     cycles, ##SUMS

.rotate         WRPIN   SetGio, #ADC_PIN  ' internal ground reference
                CALL    #measure
                ADD     Gio, MsrA
                WRPIN   SetVio, #ADC_PIN  ' internal supply reference
                CALL    #measure
                ADD     Vio, MsrA
                WRPIN   SetSig, #ADC_PIN  ' the external pin
                CALL    #measure
                ADD     Sig, MsrA
                DJNZ    cycles, #.rotate

.publish        RDLONG  pa, ptra[3]  wz   ' wait until Spin2 took it
        if_nz   JMP     #.publish
                WRLONG  Gio, ptra[0]
                WRLONG  Vio, ptra[1]
                WRLONG  Sig, ptra[2]
                WRLONG  #1, ptra[3]       ' fresh reading is ready
                JMP     #loop

' Sum eight clean SINC2 differentials from the selected source. The
' first 3 samples after a source switch are contaminated (2 for the
' SINC2 filter, 1 for the front end to settle); the 4th is clean.
measure         MOV     MsrA, #0
                MOV     pa, #3+8-1        ' 3 settling + 8 summed
.s              WAITSE1                   ' wait for the next sample
                CMP     iter, #3  wc      ' iter < 3 -> still settling
                RDPIN   pb, #ADC_PIN      ' read the SINC2 accumulator
                SUB     pb, DiffA         ' differential vs last read
                ADD     DiffA, pb         ' update running accumulator
                ZEROX   pb, #26           ' keep the 27-bit result
        if_nc   ADD     MsrA, pb          ' sum only clean samples
                INCMOD  iter, pa
        _ret_   TJNZ    iter, #.s

SetGio          LONG    P_ADC_GIO | P_ADC ' internal ground reference
SetVio          LONG    P_ADC_VIO | P_ADC ' internal supply reference
SetSig          LONG    P_ADC_1X  | P_ADC ' external pin, 1x gain

iter            LONG    0
DiffA           LONG    0

MsrA            RES     1
Gio             RES     1
Vio             RES     1
```

↪ continued from previous page

```
Sig             RES     1
cycles          RES     1


                                                    adc-single-pin-base.spin2
```

How this works, top to bottom:

- **The mailbox.** `mbox[0..2]` carry the three accumulated sums from the cog to Spin2; `mbox[3]` is a one-deep handshake flag. The cog fills the sums, sets the flag, and waits; Spin2 reads the sums, clears the flag, and computes. No torn reads, and the cog only advances when Spin2 has taken the last result.
- **The measurement engine (`measure`).** With the source already selected, it waits on the smart pin's sample-ready event (`WAITSE1`), reads the SINC2 accumulator, and takes the differential against the previous read (`SUB` / `ADD` on `DiffA`). It discards the first three differentials after the source was switched and sums the next eight into `MsrA`. The `ZEROX pb, #26` keeps the meaningful 27 bits of each result.
- **The rotation (`loop`).** For each of `SUMS` cycles, it selects GIO, then VIO, then the pin, calling `measure` for each and adding into the matching accumulator. Selecting a source is a single live WRPIN — the smart pin never stops.
- **The ratio (Spin2).** `muldiv64(pinRef - gioRef, FULLSCALE_UV, vioRef - gioRef)` performs (pin – GIO) / (VIO – GIO) × 3,300,000 with a 64-bit intermediate, so the large accumulated sums never overflow. Because the result is a ratio, the sample count in each accumulator cancels — you can change `SUMS` without rescaling anything.

★ **Tip:** `SUMS` is your one knob here. More rotations means more differentials averaged per reading — more resolution, fewer readings per second. Fewer rotations means a faster update at coarser resolution. (The recipes later in this note make that trade more flexible still, with a filter that hands you *every* rate at once.)

Running the base build

Jumper pin 33 to pin 32, build with DEBUG enabled, and run. The `uV` scope window opens and the trace settles to **about 1,650,000** — the half-scale voltage the harness is driving onto the pin. The first reading or two may sit off before the differencing filter settles; after that the trace holds steady within a noise band.

♦ **Verify:** The quickest no-instrument sanity check needs no DAC at all — move the jumper so the measured pin goes to GND, and the reading should collapse toward **0 µV**. Tie it to the board's 3.3 V rail instead and it should climb toward **3,300,000 µV**. If the reading is pinned at 0, near full scale regardless of input, or wildly noisy, work through the failure branches in the Verify section below.

Choosing a Technique

The base build covers the common case. The rest of this note is a small catalog of variations, and three questions tell you which one to reach for:

- **Resolution** — how quiet does the reading need to be?
- **Channels** — how many separate voltages are you measuring?
- **Sample rate** — how fast, and do you need to reject mains hum or read above 3.3 V?

Every recipe below is built on the same shared harness and the same three-source measurement as the base build; each changes only what it must.

If you need…	Use	What it adds
One channel, absolute microvolts	**Recipe 1** — the base build	the foundation; start here
The lowest noise in a given time, on one node	**Recipe 2** — three pins	constant impedance + 3× the data per unit time
To pick rate-versus-resolution at runtime	**Recipe 3** — the filter cascade	every averaging depth available at once
To read a voltage above 3.3 V	**Recipe 4** — range extension	an external series resistor + a scale factor
To reject 50/60 Hz mains pickup	**Recipe 5** — mains averaging	integration locked to one mains cycle
Up to eight channels at once	**The ceiling** — eight-pin interpreter	a reference design, not a starting point

Recipe 1 — Single-Pin Absolute Microvolts

This *is* the base build above. Reach for it whenever one pin, moderate resolution, and a simple program are enough. Its one tuning knob, `SUMS`, trades update rate for resolution; the recipes below trade differently.

Recipe 2 — Three Pins for 3× the Data

Use it when you are measuring one node and want the quietest reading you can get in a fixed amount of time.

The base build switches one pin among GIO, VIO, and the pin. Each switch yanks the source's load impedance around, and only one third of the time is spent actually reading the pin. Tie **three** pins to the same node and you can do better: at every instant the three pins hold the three sources in *some* rotation, so **exactly one pin is always reading the node**. The source sees a constant impedance, and all three pins contribute to the GIO, VIO, and pin accumulators — three times the data in the same wall-clock time, for correspondingly lower noise. (This is the P2 designer's

bench technique; the *amount* of improvement is board- and source-dependent, so treat "3×" as the mechanism, not a guaranteed figure.)

Tie pins 32, 33, 34, and the DAC pin 35 together to a common node. Those four pins are exactly power group P32–35, so the three measurement pins share a single VIO/GIO reference domain — keep your pins within one four-pin group if you move to other pins, or the references stop agreeing (see Pitfalls). The Spin2 harness and the ratiometric consumer are unchanged from the base build; only the pin assignments and the measurement engine differ:

```
CON
  _clkfreq = 200_000_000

  PINA = 32                  ' three ADC pins, tied together to the node
  PINB = 33
  PINC = 34
  ADC_GRP = PINA | %10_000000   ' pin group: base + 2 (three pins)
  DAC_PIN = 35               ' loopback test-signal source (also tied)

  FULLSCALE_UV = 3_300_000
  TEST_UV      = 1_650_000
  SUMS = 64

VAR
  LONG mbox[4]

PUB main() | gioRef, vioRef, pinRef, uv
  start_test_signal()
  open_scope()
  COGINIT(NEWCOG, @adc_engine, @mbox)

  repeat
    repeat until mbox[3]
    gioRef := mbox[0]
    vioRef := mbox[1]
    pinRef := mbox[2]
    mbox[3] := 0
    uv := muldiv64(pinRef - gioRef, FULLSCALE_UV, vioRef - gioRef)
    DEBUG(`uV `(uv))

PRI start_test_signal() | dac16
  dac16 := muldiv64(TEST_UV, $FFFF, FULLSCALE_UV)
  WRPIN(DAC_PIN, P_DAC_DITHER_PWM | P_DAC_124R_3V | P_OE)
  WXPIN(DAC_PIN, $100)
  PINH(DAC_PIN)
  WYPIN(DAC_PIN, dac16)

PRI open_scope()
  DEBUG(`SCOPE uV size 600 400 samples 256 rate 1 dotsize 2 pos 100 100)
  DEBUG(`uV 'uV' 0 3_300_000 360 20 15 MAGENTA)

DAT             ORG
```

continues on next page →

↪ continued from previous page

```
adc_engine
                MOV     pa, #PINA
                OR      pa, #%001_000000  ' SE1 = PINA's IN rising
                SETSE1  pa

                FLTL    #ADC_GRP          ' reset all three pins
                WRPIN   SetGio, #ADC_GRP  ' ADC mode on all three
                WXPIN   #%01_0111, #ADC_GRP   ' SINC2 filter, 128 clocks
                DIRH    #ADC_GRP          ' enable all three

loop            MOV     Gio, #0
                MOV     Vio, #0
                MOV     Sig, #0
                MOV     cycles, ##SUMS

                ' Each sub-cycle assigns the three sources to the three pins
                ' in a different rotation, so exactly one pin always reads
                ' the node -> the source sees a constant impedance.
.rotate         WRPIN   SetGio, #PINA     ' A=GIO  B=VIO  C=pin
                WRPIN   SetVio, #PINB
                WRPIN   SetSig, #PINC
                CALL    #measure
                ADD     Gio, MsrA
                ADD     Vio, MsrB
                ADD     Sig, MsrC

                WRPIN   SetVio, #PINA     ' A=VIO  B=pin  C=GIO
                WRPIN   SetSig, #PINB
                WRPIN   SetGio, #PINC
                CALL    #measure
                ADD     Vio, MsrA
                ADD     Sig, MsrB
                ADD     Gio, MsrC

                WRPIN   SetSig, #PINA     ' A=pin  B=GIO  C=VIO
                WRPIN   SetGio, #PINB
                WRPIN   SetVio, #PINC
                CALL    #measure
                ADD     Sig, MsrA
                ADD     Gio, MsrB
                ADD     Vio, MsrC
                DJNZ    cycles, #.rotate

.publish        RDLONG  pa, ptra[3]  wz
        if_nz   JMP     #.publish
                WRLONG  Gio, ptra[0]
                WRLONG  Vio, ptra[1]
                WRLONG  Sig, ptra[2]
                WRLONG  #1, ptra[3]
                JMP     #loop

' Sum eight clean SINC2 differentials from all three pins at once.
measure         MOV     MsrA, #0
                MOV     MsrB, #0
```

↪ continued from previous page

```
                MOV     MsrC, #0
                MOV     pa, #3+8-1
.s              WAITSE1
                CMP     iter, #3  wc
                RDPIN   pb, #PINA
                SUB     pb, DiffA
                ADD     DiffA, pb
                ZEROX   pb, #26
        if_nc   ADD     MsrA, pb
                RDPIN   pb, #PINB
                SUB     pb, DiffB
                ADD     DiffB, pb
                ZEROX   pb, #26
        if_nc   ADD     MsrB, pb
                RDPIN   pb, #PINC
                SUB     pb, DiffC
                ADD     DiffC, pb
                ZEROX   pb, #26
        if_nc   ADD     MsrC, pb
                INCMOD  iter, pa
        _ret_   TJNZ    iter, #.s

SetGio          LONG    P_ADC_GIO | P_ADC
SetVio          LONG    P_ADC_VIO | P_ADC
SetSig          LONG    P_ADC_1X  | P_ADC

iter            LONG    0
DiffA           LONG    0
DiffB           LONG    0
DiffC           LONG    0

MsrA            RES     1
MsrB            RES     1
MsrC            RES     1
Gio             RES     1
Vio             RES     1
Sig             RES     1
cycles          RES     1
```

adc-three-pin.spin2

♦ **Verify:** With all four pins tied and the DAC at half scale, the reading sits near 1,650,000 µV just as in the base build — but the trace should be visibly *tighter*. Drive the node to GND and it collapses toward 0 µV.

Recipe 3 — A Filter That Gives You Every Rate at Once

Use it when you don't want to commit to one rate-versus-resolution trade-off up front — different parts of your program want the same measurement at different speeds.

The base build picks a single trade with `SUMS`. This recipe instead runs the ADC flat-out (`SUMS = 1`) and feeds each fast sample into a **time-halving filter cascade**: stage 0 is the raw sample, stage 1

is the average of the last two, stage 2 the average of the last two of *those*, and so on — seventeen stages, each half the rate and roughly a half-bit deeper than the one before. A bit-reversed counter updates just one stage per period, so the whole cascade costs near-constant time and the ADC never slows down. Your consumer simply reads whichever stage it wants from a hub array.

Because the cascade runs at the full sample rate, the ratiometric divide moves into the cog (the `compute` routine, using the CORDIC solver) rather than staying in Spin2 — there is no time to round-trip each sample. The wiring is the same single pin as the base build:

```
CON
  _clkfreq = 200_000_000

  ADC_PIN = 32
  DAC_PIN = 33

  FULLSCALE_UV = 3_300_000
  TEST_UV      = 1_650_000

  SUMS  = 1                   ' run the ADC flat-out; the filter averages
  STAGE = 6                   ' which averaging stage to display (0..16)

VAR
  LONG Samples[17]            ' 17 averaging depths, filled by the cascade

PUB main()
  start_test_signal()
  open_scope()
  COGINIT(NEWCOG, @adc_engine, @Samples)
  repeat
    DEBUG(`uV `(Samples[STAGE]))      ' deeper stage = smoother, slower
    waitms(20)

PRI start_test_signal() | dac16
  dac16 := muldiv64(TEST_UV, $FFFF, FULLSCALE_UV)
  WRPIN(DAC_PIN, P_DAC_DITHER_PWM | P_DAC_124R_3V | P_OE)
  WXPIN(DAC_PIN, $100)
  PINH(DAC_PIN)
  WYPIN(DAC_PIN, dac16)

PRI open_scope()
  DEBUG(`SCOPE uV size 600 400 samples 256 rate 1 dotsize 2 pos 100 100)
  DEBUG(`uV 'uV' 0 3_300_000 360 20 15 MAGENTA)

DAT             ORG

adc_engine
                MOV     SamplePtr, ptra    ' hub base of the 17-stage array
                MOV     pa, #ADC_PIN
                OR      pa, #%001_000000
                SETSE1  pa
                FLTL    #ADC_PIN
                WRPIN   SetSig, #ADC_PIN
```

continues on next page →

↪ continued from previous page

```
                WXPIN   #%01_0111, #ADC_PIN
                DIRH    #ADC_PIN

loop            MOV     Gio, #0
                MOV     Vio, #0
                MOV     Sig, #0
                MOV     cycles, ##SUMS

.rotate         WRPIN   SetGio, #ADC_PIN
                CALL    #measure
                ADD     Gio, MsrA
                WRPIN   SetVio, #ADC_PIN
                CALL    #measure
                ADD     Vio, MsrA
                WRPIN   SetSig, #ADC_PIN
                CALL    #measure
                ADD     Sig, MsrA
                DJNZ    cycles, #.rotate

                CALL    #compute          ' Gio/Vio/Sig -> one uV sample
                CALL    #filter           ' fan it out across the 17 stages
                JMP     #loop

measure         MOV     MsrA, #0
                MOV     pa, #3+8-1
.s              WAITSE1
                CMP     iter, #3  wc
                RDPIN   pb, #ADC_PIN
                SUB     pb, DiffA
                ADD     DiffA, pb
                ZEROX   pb, #26
        if_nc   ADD     MsrA, pb
                INCMOD  iter, pa
        _ret_   TJNZ    iter, #.s

' Ratiometric uV via CORDIC: r = ABS(Sig-Gio)<<32 / ((Vio-Gio)<<1),
' then sample = r * (3_300_000<<1) >> 32. ABS+<<1 admit Sig<Gio and Sig>Vio.
compute         MOV     pa, Sig
                SUB     pa, Gio
                ABS     pa  wc
                MOV     pb, Vio
                SUB     pb, Gio
                SHL     pb, #1
                QFRAC   pa, pb
                GETQX   pa
                QMUL    pa, ##3_300_000<<1
                GETQY   pa
        if_c    NEG     pa
        _ret_   MOV     sample, pa

' N-stage time-halving filter. Stage 0 is the raw sample; stage N is the
' average of two stage-(N-1) samples, emitted every 2^N periods. A bit-
' reversed counter picks which one stage to update this period, so the whole
' cascade costs near-constant time and the ADC never slows down.
```

↪ continued from previous page

```
filter          MOV     fb+0, fa+0        ' shift sample into stage 0
                MOV     fa+0, sample
                WRLONG  sample, SamplePtr ' stage 0 written every period

                ADD     fctr, #1          ' choose the stage to update
                MOV     fenc, fctr
                REV     fenc
                ENCOD   fenc
                SUBR    fenc, #31
                CMP     fenc, #16  wc
        if_nc   RET                       ' nothing due this period

                ALTS    fenc, #fa+1       ' shift old sample within stage N
                MOV     pa, 0-0
                ALTD    fenc, #fb+1
                MOV     0-0, pa
                ALTS    fenc, #fa+0       ' add the two stage-(N-1) samples
                MOV     pa, 0-0
                ALTS    fenc, #fb+0
                ADD     pa, 0-0
                ALTD    fenc, #fa+1       ' save the new stage-N sample
                MOV     0-0, pa
                SAR     pa, fenc          ' divide by 2^(N+1)
                SAR     pa, #1  wc
                ADDX    pa, #0
                ADD     fenc, #1          ' write stage N to hub
                SHL     fenc, #2
                ADD     fenc, SamplePtr
        _ret_   WRLONG  pa, fenc

SetGio          LONG    P_ADC_GIO | P_ADC
SetVio          LONG    P_ADC_VIO | P_ADC
SetSig          LONG    P_ADC_1X  | P_ADC

iter            LONG    0
DiffA           LONG    0
fctr            LONG    0

SamplePtr       RES     1
MsrA            RES     1
Gio             RES     1
Vio             RES     1
Sig             RES     1
cycles          RES     1
sample          RES     1
fenc            RES     1
fa              RES     17
fb              RES     17
```

adc-filter-cascade.spin2

Change `STAGE` to choose your trade-off without touching the ADC at all: low stages update fast and jitter more; high stages update slowly and sit rock-steady. Two cogs could even read *different* stages of the *same* live measurement.

♦ **Verify:** With `STAGE = 6` the reading sits near 1,650,000 µV and is smoother than the base build's. Set `STAGE = 0` and the same trace gets noticeably noisier (you're watching the raw, unaveraged sample); set `STAGE = 12` and it crawls but is very steady.

Recipe 4 — Reading a Voltage Above 3.3 V

Use it when the voltage you want to measure exceeds the pin's 0–3.3 V window.

The technique is mostly *hardware*: put a large resistor in series between the source and the pin. That resistor and the pin's own input resistance form a divider, so only a safe fraction of the source voltage reaches the pin. The code then scales the pin reading back up by the divider ratio.

The catch is that the pin's input resistance is not a published figure — no Parallax source states one — and it varies part to part. So there is no datasheet resistance to compute the ratio from: **calibrate** it instead, applying one known source voltage and adjusting the ratio until the reading is true. After that, the only change to the base build is the CON divider constants, a wider scope range, and one extra scaling line in the consumer:

```
CON
  ' External series resistor forms a divider with the pin's input
  ' resistance. Treat the ratio as a CALIBRATION constant: apply one
  ' known source voltage and adjust DIV_NUM/DIV_DEN until it reads true.
  ' R_IN is not a published figure and varies, so these are only a
  ' starting point -- calibrate, do not compute.
  DIV_NUM = 9_500_000        ' R_EXT + R_IN  (starting point)
  DIV_DEN =   500_000        ' R_IN          (starting point)
  SOURCE_FS_UV = 62_700_000  ' extended full scale = 3.3V * DIV_NUM/DIV_DEN
```

```
    uv := muldiv64(pinRef - gioRef, FULLSCALE_UV, vioRef - gioRef)
    uv := muldiv64(uv, DIV_NUM, DIV_DEN)      ' undo the external divider
    DEBUG(`uV `(uv))
```

and widen the scope to the new full scale:

```
  DEBUG(`uV 'uV' 0 62_700_000 360 20 15 MAGENTA)
```

■ **Hardware:** Match the external resistor's temperature coefficient to the on-chip resistor, or the ratio (and your calibration) will drift with temperature. Add a small capacitor from the pin to GIO to shunt high-frequency spikes. For AC-only measurements, a series *capacitor* lets the pin self-bias while still passing the signal.

♦ **Verify:** Apply a known voltage above 3.3 V through the series resistor and confirm the reading matches your meter after calibration. Without an external resistor in place, this build's loopback reads the DAC voltage multiplied by the divider ratio — expected, since there is then no divider to undo.

Recipe 5 — Averaging Out Mains Hum

Use it when your signal is contaminated by 50/60 Hz pickup — long leads, a transformer nearby, a high-impedance source acting as an antenna.

The fix is free: make each reading integrate over **exactly one mains cycle**. A full cycle of 50/60 Hz pickup sums to zero, so it simply disappears from the result. The only change is to compute `SUMS` from the mains frequency instead of hard-coding it:

```
CON
  MAINS_HZ = 60              ' set 50 where mains is 50 Hz
  ' One rotation = 3 sources x (3 settle + 8 summed) samples x 128 clocks.
  CLOCKS_PER_ROT = 3 * 11 * 128
  ' Sum exactly one mains period so the 50/60 Hz pickup integrates to zero.
  SUMS = _clkfreq / (MAINS_HZ * CLOCKS_PER_ROT)
```

Everything else is the base build unchanged. At 200 MHz this makes `SUMS` about 789 rotations for 60 Hz (947 for 50 Hz), and the reading updates once per mains cycle.

♦ **Verify:** Hold an un-averaged base-build reading next to this one on a noisy bench (touch the input lead). The mains-averaged trace should be dramatically calmer, its residual ripple a fraction of the un-averaged version's.

The Ceiling — Eight Channels via a Bytecode Interpreter

The techniques above scale to a handful of pins by running more measurement cogs. To read **eight** instrumentation channels from a *single* cog, the P2 designer wrote something more ambitious: a tiny bytecode interpreter in PASM2 whose "program" is a list of ADC operations (reset, add-pin, start, select-GIO, measure, average, wait, jump) stored in hub. A `SKIP`-pattern core handles a variable number of pins from the same code, and each channel keeps its own running average.

This note does not re-teach that design — it is a *reference* for what the ceiling looks like, not a starting point. The complete source (`EightPinADC.spin2`) and the discussion that produced it are in the forum thread cited in the References. If you have understood the base build, you have the foundation that design is built on; the rest is engineering for density, not new ADC theory.

See It Work / Verify

Every recipe shares the same success signal and the same cheap sanity check, so one verification routine covers them all.

What success looks like. With the DAC loopback driving half scale, the `uV` scope trace settles near **1,650,000 µV** and holds within a noise band whose width depends on the technique. The first reading or two may sit off while the differencing filter settles; that is normal.

The no-instrument self-check. Move the measured pin to GND — the reading collapses toward **0 µV**. Move it to the 3.3 V rail — it climbs toward **3,300,000 µV**. Above the supply reference the reading exceeds full scale in every build. Below the ground reference the behavior depends on the math: the CORDIC build (Recipe 3) carries the sign explicitly and reads *negative*, while the `muldiv64` builds (the base build and Recipes 1, 2, 4, and 5) take an **unsigned** ratio, so a pin driven below ground does not read negative — the difference wraps and the reading pegs off-scale high. In every case an out-of-band input producing an out-of-band reading is correct.

When it doesn't work:

- **Reading pinned at 0, or wildly large/erratic.** The jumper is almost certainly missing or on the wrong pin, so the ADC pin floats. Confirm the wiring first.
- **Reading stuck near full scale regardless of input.** `VIO - GIO` came out near zero, so the ratio blew up. Check that the source select constants are right (`P_ADC_GIO`, `P_ADC_VIO`, `P_ADC_1X`) and that the pin is actually in ADC mode.
- **Reading is in the right ballpark but noisy.** Expected on a breadboard. Raise `SUMS` (Recipe 1/2), read a deeper `STAGE` (Recipe 3), or lock to the mains cycle (Recipe 5). If it is *absolutely* off by a stubborn offset that averaging won't fix, you are seeing the matched-resistor error floor — see Pitfalls.
- **Nothing on the DEBUG window.** Build with DEBUG enabled (`pnut-ts -d`); without it the `DEBUG()` directives are ignored.

Pitfalls & Notes

▲ **Pitfall — the matched-resistor error floor.** GIO, VIO, and the pin are each measured through a *separate* on-chip resistor, and those resistors do not match perfectly, so absolute accuracy has a floor that averaging cannot reach past. Measured on P2 silicon (2026-07-07), the single-pin ratiometric absolute error was **≤9 mV**, reproducible, appearing as a small positive offset at low voltages and near zero at mid and high scale. Because it is an offset rather than noise, **no amount of averaging removes it.** Where absolute accuracy matters, calibrate each pin once against a known voltage (or drive the pin to each rail and measure the result).

The P2's designer separately reports having seen pins read as much as 15 mV apart *pin-to-pin* (Reference 2). Treat that as a designer report, not a specification — the bench has not yet reproduced it, and the multi-pin measurement that would settle it remains open. Design to the measured ≤9 mV single-pin floor, and do not quote 15 mV as a specification. See the I/O & Smart Pins User Guide §16.8 for the front-end limits and the calibration guidance.

▲ **Pitfall — high-impedance sources load the pin.** The ADC input is not a high-impedance node: it presents a finite resistance that no Parallax source specifies. A high-impedance source forms

a divider with it and reads low. Buffer such sources, or measure the divider and account for it as in Recipe 4.

■ **Hardware — feed VIO from a clean supply.** The internal references track the VIO rail, so a noisy switch-mode VIO degrades every reading. For instrumentation work, supply VIO from a clean LDO.

■ **Hardware — keep a multi-pin measurement inside one power group.** The P2 powers its I/O pins in **isolated groups of four** — pins 0–3, 4–7, ..., 60–63 — and each group shares one VIO/GIO supply pair. A pin's ADC references *its own group's* VIO and GIO, which is exactly what makes the single-pin ratiometric measurement absolute. It also means the pins you tie together for one measurement — Recipe 2's three pins, or any layout that shares a node across pins — should all sit **within a single group**, so they share one reference domain. Straddling a boundary mixes supply domains and degrades the result. (That is why these examples use pins 32–35: those four pins are exactly one group.)

The boards suggest a different number. On a P2 Edge module the I/O headers come in **eights**, each with its own 3.3 V regulator, which makes the chip's power domains look eight pins wide. They are not: one Edge regulator feeds **two** of the chip's four-pin domains. Those two domains share a supply *net* on the board, but each still reaches the die through its own VIO pin and its own bond wire, so a little supply difference survives between them. Grouping by four is therefore the rule that holds on any board, and the one to design to. The eight-pin figure is still the one you want for **current budgeting** on an Edge — 300 mA per header group. The mechanism is owned by the P2 I/O & Smart Pins User Guide, Chapter 16; the grouping is in the P2 datasheet pin descriptions and the package pinout.

★ **Tip — the sample period is adjustable.** These builds run SINC2 *filtering* mode, which — unlike SINC2 *sampling* mode — does **not** restrict the period to a power of two. `X[3:0]` sets a convenient power-of-two default (`%0111` = 128 clocks), but a `WYPIN` issued after the `WXPIN` overrides it with any period up to about 11,585 clocks. A longer period trades sample rate for per-sample resolution.

▲ **Pitfall — stay inside the clock the datasheet states.** These builds run at 200 MHz. The datasheet's AC characteristics give the PLL system clock as **3.33 MHz minimum, 180 MHz typical, 320 MHz maximum**, with the footnote that the nominal 180 MHz figure holds up to 105 °C. The original research code's 320 MHz therefore sat *at* the datasheet maximum rather than beyond it, though well above the typical rating and dependent on cooling. Either way the ADC works at any clock from about 10 MHz up — reach for integration time, not clock rate, when you want resolution.

★ **Tip — the loopback is a functional check, not a benchmark.** The on-chip DAC and ADC share the same die and the same imperfections, so a loopback proves your *code* runs, not that your *absolute accuracy* is good. Characterize accuracy against an external precision reference and meter.

Conclusion

The idea under every recipe in this note is the same one: a P2 pin ADC reading is only a comparison against the chip's own drifting references, so to get an *absolute* voltage you measure those references too and take a ratio. Once that single measurement exists — the base build — everything else is a variation on how you spend time and pins: three pins for lower noise, a filter cascade to defer the rate decision, a series resistor to reach higher voltages, a mains-locked window to erase hum. The eight-channel interpreter is the same idea pushed to its engineering limit. Start from the base build, pick the variation your project needs, and verify it with a jumper to GND before you trust a single microvolt.

Resources

- **Example code (download).** The worked builds in this note — the base instrumentation ADC, the three-pin variant, and the filter cascade — are published together as the **P2AN001 source ZIP** beside this note's PDF, each a complete `pnut-ts`-compiled program ready to build and run.
- **P2 I/O & Smart Pins User Guide, Chapter 16 (ADC)** (a companion P2 Knowledge Base publication) — the mechanism this note applies: the sigma-delta front end, the SINC filters, the gain ladder, the internal references, and the ratiometric method (§16.3) and accuracy limits (§16.8).
- **"Improved ADC Pin Techniques"** — the Parallax Forums discussion in which the P2 designer developed and published these techniques, including the complete eight-channel source. See the References.

References

These are the authoritative primary sources the note's facts trace to. (The companion *P2 I/O & Smart Pins User Guide* named in Resources derives from these same sources.)

1. *Propeller 2 Documentation v35 — Rev B/C Silicon* (Chip Gracey, Parallax Inc.) — Smart Pins, ADC modes %11000/%11001/%11010: the sigma-delta front end, the SINC filters, and the ratiometric method.
2. Chip Gracey et al., "Improved ADC Pin Techniques," Parallax Forums, 2023–2024. `https://forums.parallax.com/discussion/175609/improved-adc-pin-techniques`
3. *Spin2 Language Documentation* (Chip Gracey, Parallax Inc.) — `MULDIV64`, smart-pin configuration (`WRPIN`/`WXPIN`/`WYPIN`).

Revision History

The example library `P2AN001-src.zip` is versioned with the note, so a ZIP downloaded beside this PDF matches the code printed here. This note's version is on its cover, and the top row below.

Version	Date	In brief
1.0.6	2026-11-05	Names in capitals stay whole at a line end
1.0.5	2026-09-10	The clock ceiling, the measured error floor, and the unpublished input resistance
1.0.4	2026-08-16	I/O power-domain grouping, and the source attribution for the error floor
1.0.3	2026-08-08	Licensed CC BY-SA 4.0
1.0.2	2026-07-11	Technical-precision pass on the ADC recipes
1.0.1	2026-07-03	Editorial correction to the compile-status note
1.0.0	2026-07-03	First community-review release

What changed in each version, and why, is the note's **changelog** — published beside this PDF in the P2 Knowledge Base document catalog. That record is the authority; this table exists so a reader holding only the PDF can tell which revision they have.

Copyright & License

Copyright © 2026 Iron Sheep Productions, LLC and Parallax Inc.

This work is licensed under the Creative Commons Attribution–ShareAlike 4.0 International License (CC BY-SA 4.0) — you may share and adapt it, including commercially, with attribution and under the same terms. To view the full license, visit `https://creativecommons.org/licenses/by-sa/4.0/`.

Parallax, Propeller, and Spin are trademarks of Parallax Inc. This license grants permissions under copyright only; it does not grant rights to use these trademarks, and adapted or redistributed copies must not imply endorsement by, or official status with, Iron Sheep Productions, LLC or Parallax Inc.

Acknowledgments

- **Parallax Inc.** — for the Propeller 2 and the reference documentation that forms the foundation of this work.
- **Chip Gracey** — for the design of the P2 smart-pin ADC and the "Improved ADC Pin Techniques" work this note builds on.
- **The P2 community** — the Parallax Forums discussion that developed and verified these single-pin instrumentation techniques.